

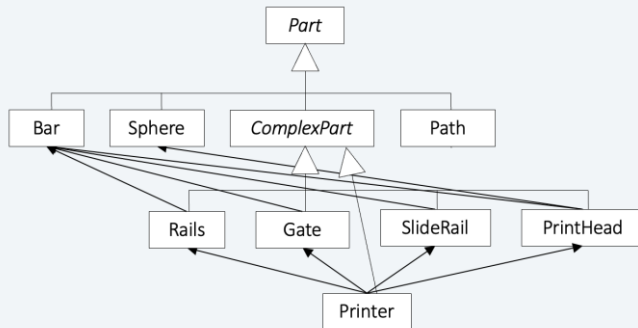
11 Objektorientierter Entwurf

11.1 Grundlegendes u. UML

11.2 Basismodell für oo Systementwurf

11.3 Entwurfsbeispiel 3D-Drucker

11.4 Entwurfsmuster



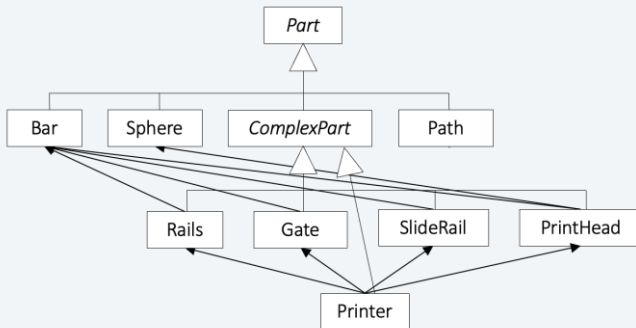
11 Objektorientierter Entwurf

11.1 Grundlegendes u. UML

11.2 Basismodell für oo Systementwurf

11.3 Entwurfsbeispiel 3D-Drucker

11.4 Entwurfsmuster



- der objektorientierte Entwurf erfordert eine andere Denkweise als der aufgabenorientierte Entwurf
- die Daten zusammen mit den darauf auszuführenden Operationen (Objekte) stehen im Mittelpunkt des Interesses
- daher werden anstatt der Algorithmen zuerst die Abstraktionen von Objekten (Klassen) und die zwischen ihnen bestehenden Beziehungen modelliert

Typische Vorgehensweise:

1. Auffinden von geeigneten Klassen zur Lösung der Aufgabe
2. Festlegen, welche Operationen mit einzelnen Objekten möglich sein sollen
3. Festlegen, in welcher Beziehung diese Klassen und ihre Objekte zueinander stehen

- Entwerfen ist ein kreativer Prozess (analytische/intuitive/kritische Phase)
- ein guter Systementwurf erfordert eine gründliche Problemanalyse
- es gibt keine allgemein anerkannte Methode für guten Systementwurf

Richtlinien

- für die Identifikation von Klassen, Datenkomponenten und Methoden
- für die Gestaltung von Klassenschnittstellen und Klassenbeziehungen

Entwurfsmuster

- für die Gestaltung von Systemarchitekturen und guten Programmstrukturen

Notation

- als allgemein akzeptierte **Notation** für den Entwurf von objektorientierten Systemen hat sich **UML (Unified Modeling Language)** durchgesetzt

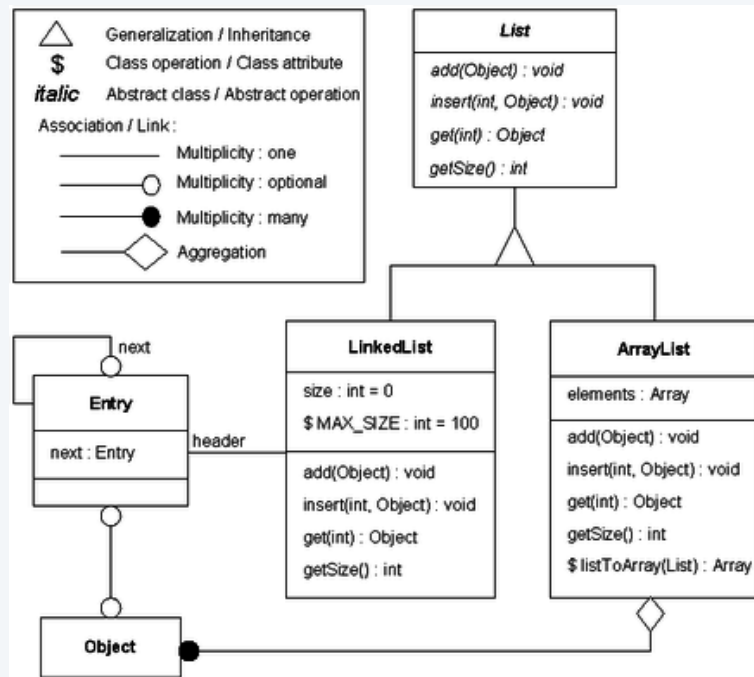
Unified Modeling Language (UML)

UML ist eine 1997 **standardisierte** Notation, die aus OMT (James Rumbaugh), OOSE (Ivar Jacobson) und den Methoden von Booch entstanden ist (die drei Amigos)

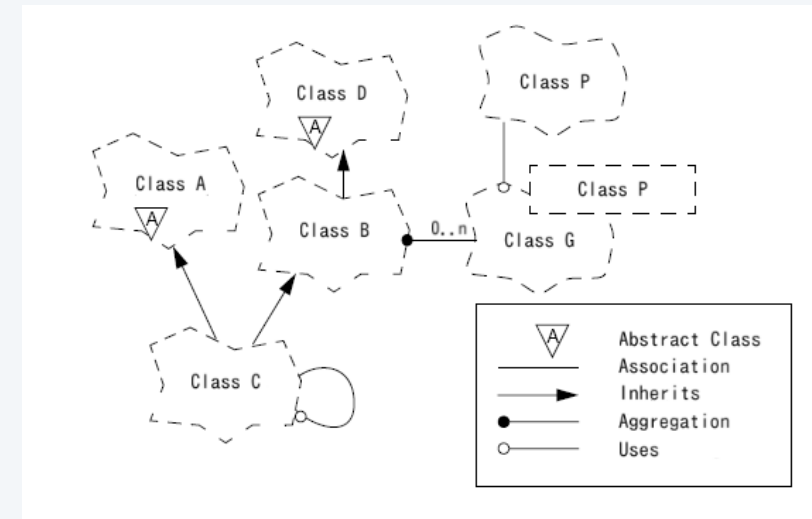


die drei Amigos

Beispiele: Klassendiagramme in OMT und Booch Method



Object-Modeling Technique (OMT)



Booch method

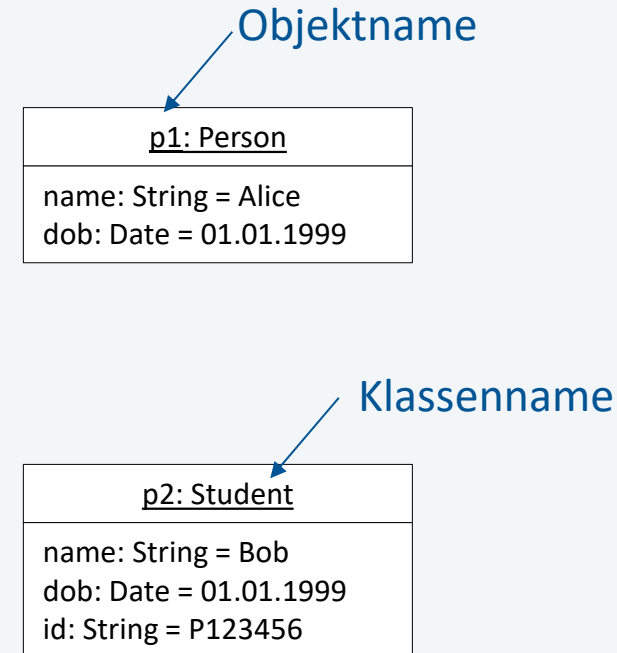
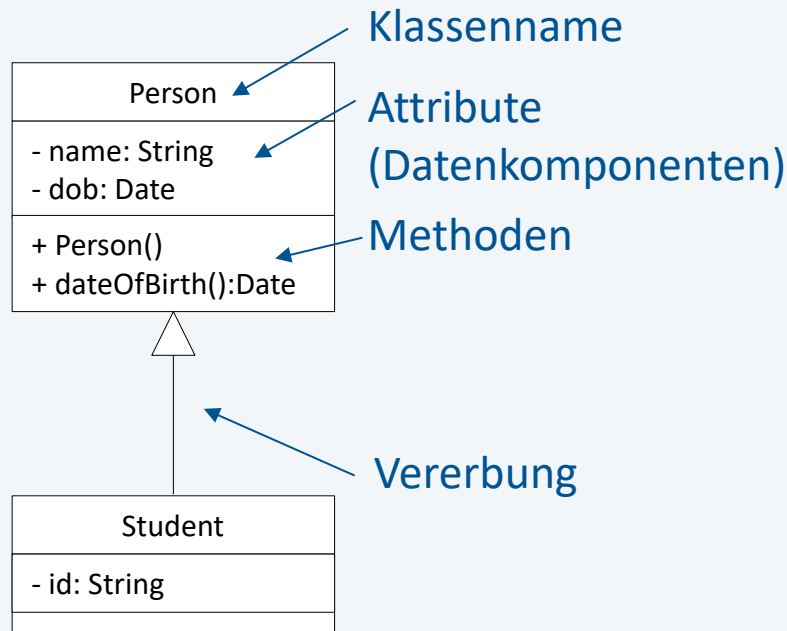
Bildquellen:

https://de.wikipedia.org/wiki/Object-Modeling_Technique

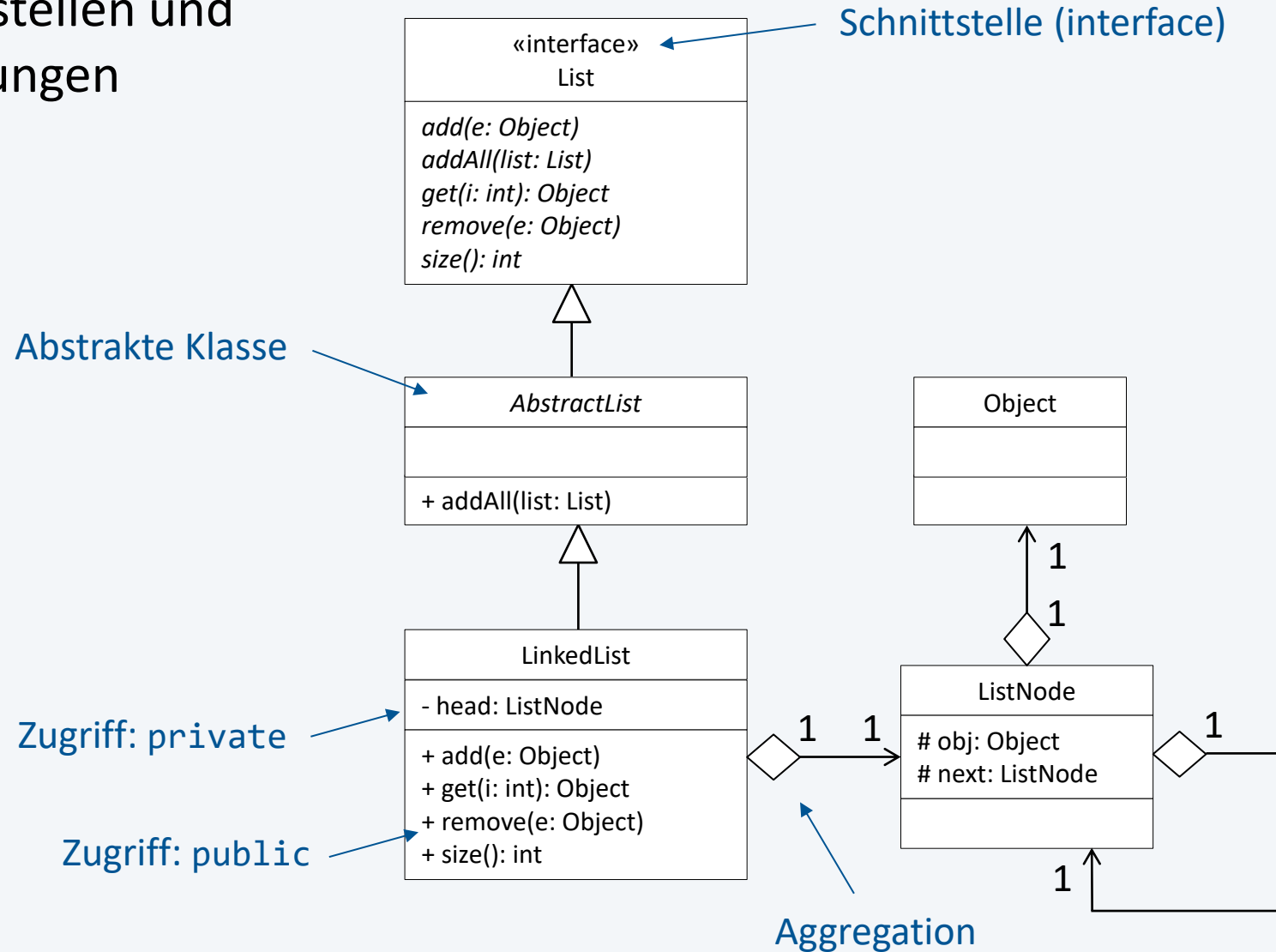
https://en.wikipedia.org/wiki/Booch_method#/media/File:Booch-diagram.png

<https://www.oose.de/blogpost/20-jahre-uml>

Klassen- und Objektdiagramme



Schnittstellen und Beziehungen



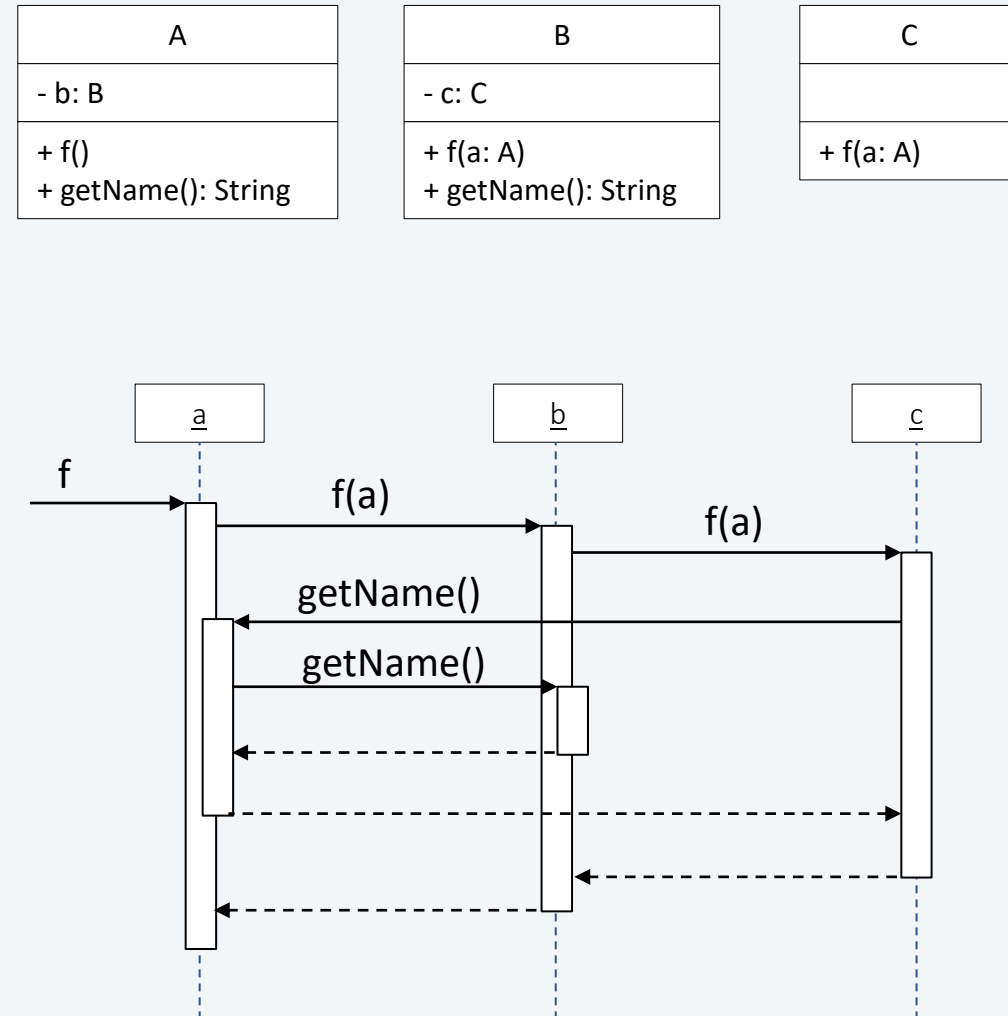
Sequenzdiagramm

```
A a = new A();
```

```
public class A {  
    private B b;  
    public void f() {  
        b.f(this);  
    }  
    public String getName() {  
        return "A" + b.getName();  
    }  
}
```

```
public class B {  
    private C c;  
    void f(A a) {  
        c.f(a);  
    }  
    String getName() {  
        return ...  
    }  
}
```

```
public class C {  
    public void f(A a) {  
        IO.println(a.getName());  
    }  
}
```



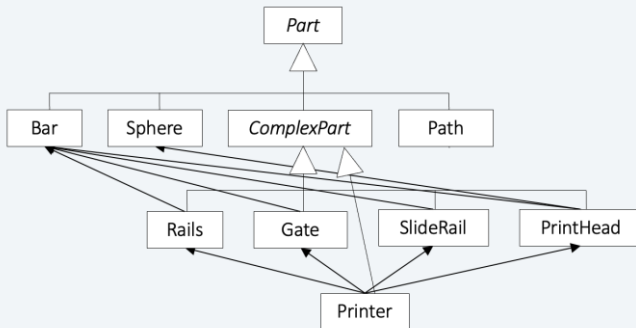
11 Objektorientierter Entwurf

11.1 Grundlegendes u. UML

11.2 Basismodell für oo Systementwurf

11.3 Entwurfsbeispiel 3D-Drucker

11.4 Entwurfsmuster



Vorgehen (in Anlehnung an Abbott)

1. **Verbale Spezifikation** der Aufgabe aufstellen (besser noch ein Prototyp)
2. Klassen, Methoden und Datenkomponenten identifizieren
 - 2.1 **Hauptwörter** sind mögliche Kandidaten für **Klassen**
 - 2.2 **Zeitwörter** sind mögliche Kandidaten für **Methoden**
 - 2.3 **Eigenschaftswörter** sind mögliche Kandidaten für **Datenkomponenten**
3. Suche nach geeigneten **Basisklassen** bzw. **Faktorisierung** von Gemeinsamkeiten
4. Spezifikationen der Klassen aufstellen
5. Algorithmischer Entwurf der **Methoden** (ggf. mit schrittweiser Verfeinerung)

Tipp: Immer etwas allgemeiner bleiben als nötig (um zu möglichst wiederverwendbaren Klassen zu kommen)

Aufgabe zur Illustration der Entwurfsmetapher

Ein Roboter soll auf einer geraden Linie von der Position p1 zur Position p2 zwei Karosserieteile durch Punktschweißung verbinden.



Bildquelle: <https://www.kuka.com/>

Konventioneller Entwurf

- Man geht von den auszuführenden Aktionen aus, hier: „durch Punktschweißung verbinden“
- Ergebnis: eine Prozedur

```
JoinBySpotWelding(↓part1, part2: BodyPart ↓p1, p2: Position)
```

Konsequenzen:

- breite Schnittstellen
- schlechte Wiederverwendbarkeit

Objektorientierter Entwurf – Vorgehen in Anlehnung nach Abbott

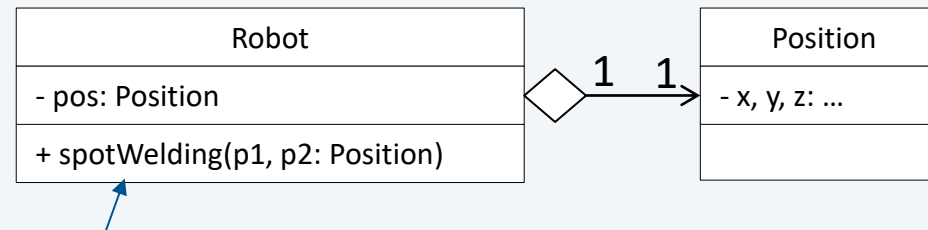
■ Entwurfsschritt 1

Ein Roboter soll auf einer geraden Linie von der Position p1 zur Position p2 zwei Karosserieteile durch Punktschweißung verbinden.

■ Entwurfsschritte 2.1 – 2.3

Hauptwörter: Roboter, Linie, Position, Karosserieteil, Punktschweißung
Zeitwörter: verbinden

■ Entwurfsschritte 3 und 4

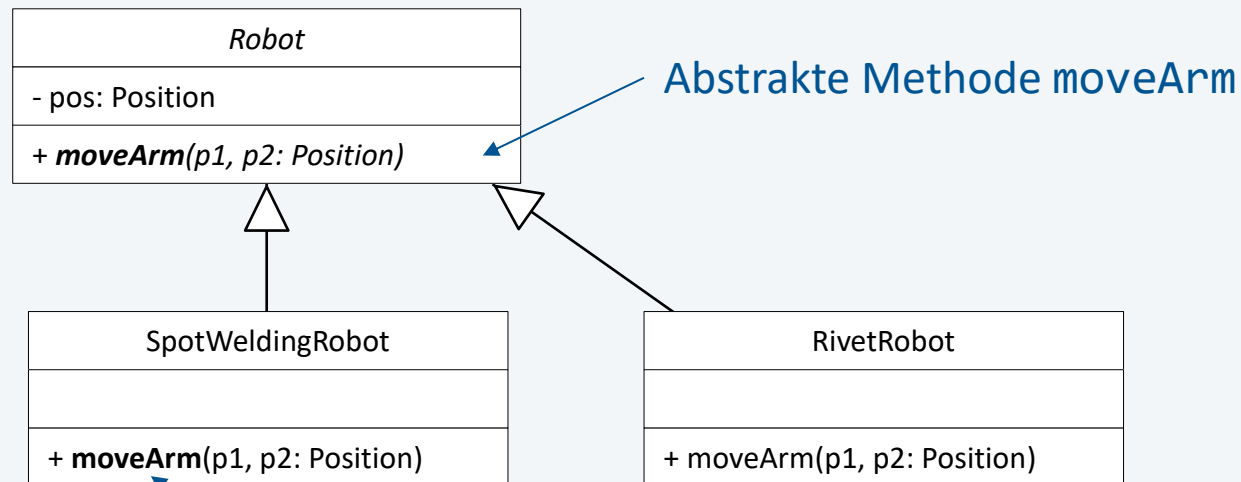


Methode `spotWelding` implementiert "durch Punktschweißung verbinden".

- Ergebnis im Wesentlichen wie bei konventionellem Entwurf
- Erweiterungen, z.B. durch Nieten verbinden, bohren, lackieren, erfordern Änderungen in der Klasse Robot

Entwurfsschritte 3 und 4

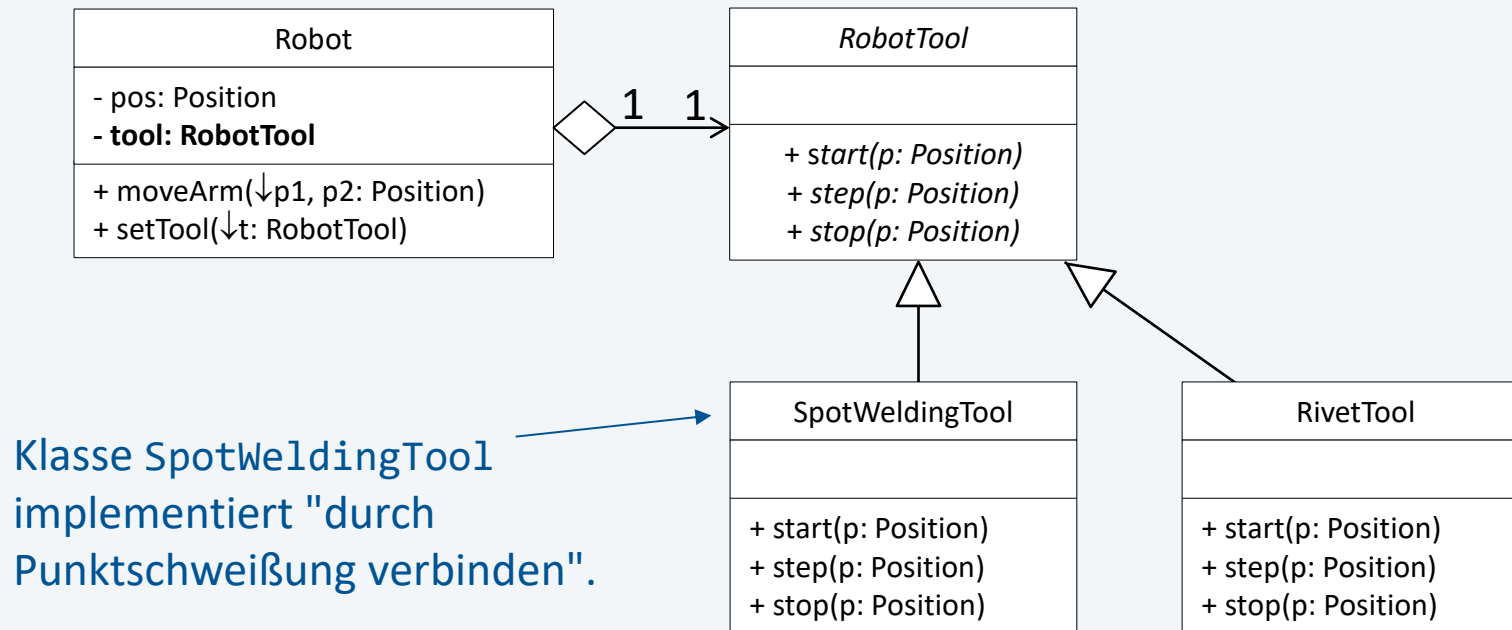
- Wunsch: Erweiterungen, ohne bestehende Klassen zu ändern
- Lösung: Erweiterung durch **Vererbung**



Methode `moveArm` implementiert "durch Punktschweißung verbinden".

Entwurfsschritte 3 und 4

- Wunsch: Erweiterungen, ohne bestehende Klassen zu ändern
- Lösung: Erweiterung durch **Aggregation**
"ein Roboter hat ein Werkzeug, z.B. für Punktschweißung"





Vorschlag von ChatGPT

1. Verbale Spezifikation / Prototyp

Ein Roboter erhält zwei Positionen `p1` und `p2`, zwischen denen er sich auf einer geraden Linie bewegt. Entlang dieser Strecke sollen zwei übereinanderliegende Karosserieteile miteinander verbunden werden. Die Verbindung erfolgt durch Punktschweißung. Während der Bewegung führt der Roboter regelmäßig Punktschweißungen durch.

Prototypischer Ablauf:

- "Roboter startet bei `p1`."
- "Er bewegt sich in Richtung `p2`."
- "In regelmäßigen Abständen wird ein Schweißpunkt gesetzt."
- "Bei `p2` endet der Vorgang."

2. Identifikation von Klassen, Methoden und Datenkomponenten

2.1 Hauptwörter (→ Klassenkandidaten)

- Roboter
- Position
- Karosserieteil
- Punktschweißung
- Linie

2.2 Zeitwörter (→ Methoden)

- bewegen
- verbinden
- schweißen
- starten
- setzen (einen Schweißpunkt setzen)

2.3 Eigenschaftswörter (→ Attribut. ↓)

```
class Roboter {
    Position aktuellePosition;
    Punktschweißgerät werkzeug;

    void bewegeZu(Position ziel) {
        aktuellePosition = ziel;
        // Simulation der Bewegung
    }

    void fuehreVerbindungDurch(Position start, Position ende, double abs
        aktuellePosition = start;
        double gesamtAbstand = start.abstandZu(ende);
        int anzahlSchweißpunkte = (int) (gesamtAbstand / abstand);
        for (int i = 0; i <= anzahlSchweißpunkte; i++) {
            double faktor = (double) i / anzahlSchweißpunkte;
            Position punkt = start.interpoliere(ende, faktor);
            bewegeZu(punkt);
            werkzeug.setzeSchweißpunkt(punkt);
        }
    }
}
```

Probleme

- nicht alle Hauptwörter haben die Bedeutung von Klassen
„Punktschweißung“, beschreibt ein Verfahren, könnte also auch zu einer Methode werden
- Zuordnung der Methoden zu Klassen kann nicht immer eindeutig vorgenommen werden
- Detailliertheitsgrad der Spezifikation hat großen Einfluss auf den daraus entstehenden Entwurf
- Gefahr zu vieler Klassen

↪ Das Verfahren darf nicht blind angewandt werden.

↪ Das Auffinden der relevanten Wörter setzt Erfahrung voraus.

Weiteres Vorgehen

- einzelne Klassen unabhängig von der Problemstellung entwickeln
- beim Detailentwurf der Klassen kommt man auf neue Klassen (iterativ)

Kategorisierung von Klassen

- aus der Spezifikation abgeleitete anwendungsnahe Klassen (z.B. Robot)
- mit der Spezifikation eng verbundene Attributklassen bzw. Klassen für die Manipulation von Datenkomponenten (z.B. Position)
- für die Implementierung benötigte Klassen (z.B. Point)

Vorteile

- Abstraktionslücke zwischen Problemwelt und Implementierung wird durch zunehmend konkretere Klassen geschlossen
- Unabhängige Planung der Funktionalität einzelner Klassen zwingt zu größerer Allgemeinheit und verbessert damit die Wiederverwendbarkeit

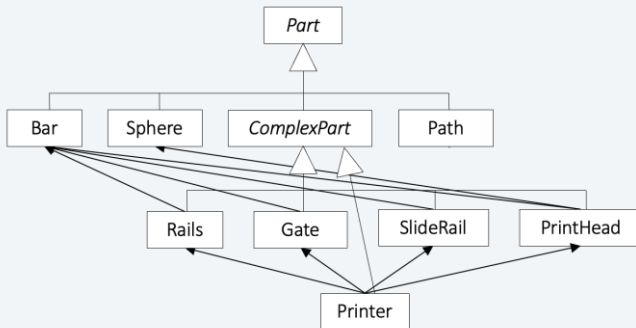
11 Objektorientierter Entwurf

11.1 Grundlegendes u. UML

11.2 Basismodell für oo Systementwurf

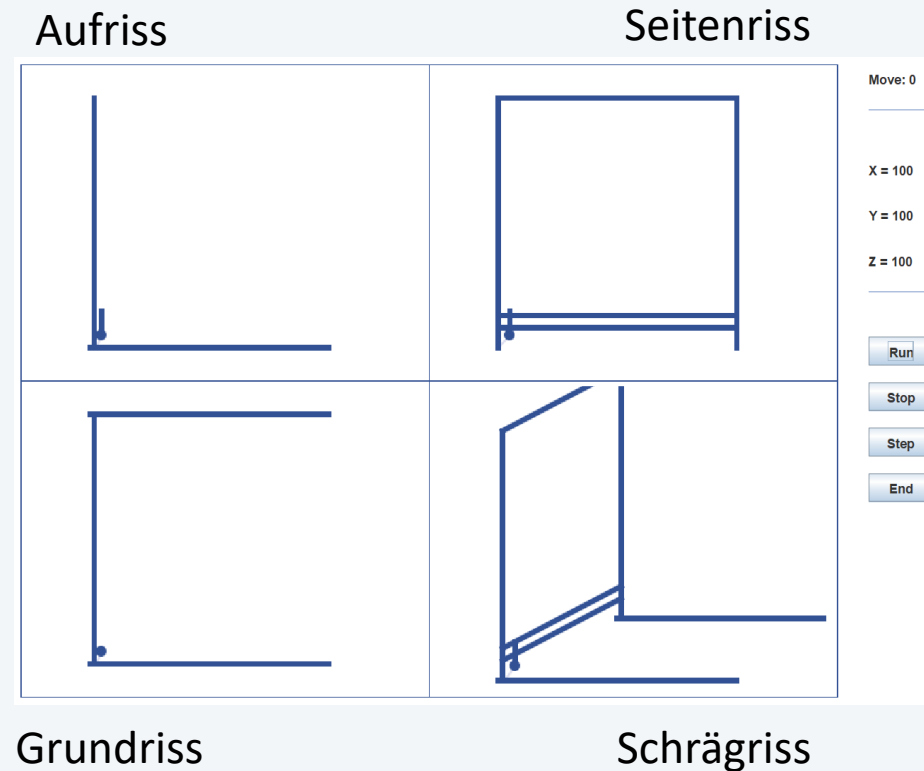
11.3 Entwurfsbeispiel 3D-Drucker

11.4 Entwurfsmuster



Zu Simulations- und Bedienungszwecken soll eine Anwendung für einen 3D-Drucker entwickelt werden

Illustration durch einen Prototyp:



Entwurfsschritt 1:

- Verbale Spezifikation der Aufgabe aufstellen bzw. Prototyp heranziehen

Spezifikation

Ein 3D-Drucker, bestehend aus einem Fundament, einem Portal und einer horizontalen Gleitschiene mit einem Druckkopf, soll lineare Bewegungen ausführen können und nach jeder Teilbewegung im Aufriss, Grundriss, Seitenriss und Schrägriss gezeichnet werden. Das Fundament besteht aus zwei Schienen, das Portal aus drei Balken (zwei vertikalen und einem horizontalen). Die horizontale Gleitschiene besteht aus zwei Balken und der Druckkopf soll durch einen Balken und eine Kugel (für die Düse) dargestellt werden. Der zurückgelegte Weg soll durch einen Polygonzug in allen vier Sichten angezeigt werden.

Entwurfsschritt 2:

- Klassen, Methoden und Attribute (Datenkomponenten) identifizieren

Spezifikation

Ein **3D-Drucker**, bestehend aus einem **Fundament**, einem **Portal** und einer horizontalen **Gleitschiene** mit einem **Druckkopf**, soll lineare Bewegungen ausführen können und nach jeder **Teilbewegung** im **Aufriss**, **Grundriss**, **Seitenriss** und **Schrägriss** gezeichnet werden. Das Fundament besteht aus zwei **Schienen**, das **Portal** aus drei **Balken** (zwei vertikalen und einem horizontalen). Die horizontale Gleitschiene besteht aus zwei Balken und der Druckkopf soll durch einen Balken und eine **Kugel** (für die Düse) dargestellt werden. Der zurückgelegte **Weg** soll durch einen **Polygonzug** in allen vier **Sichten** angezeigt werden.

Entwurfsschritt 2:

- Klassen, Methoden und Attribute (Datenkomponenten) identifizieren

Klassenkandidaten

3D-Drucker	Druckkopf	<i>Seitenriss</i>	Kugel
Fundament	<i>Teilbewegung</i>	<i>Schrägriss</i>	Weg
Portal	<i>Aufriss</i>	Schienen	<i>Polygonzug</i>
Gleitschiene	<i>Grundriss</i>	Balken	<i>Sichten</i>

Methodenkandidaten

Bewegung ausführen
gezeichnet werden

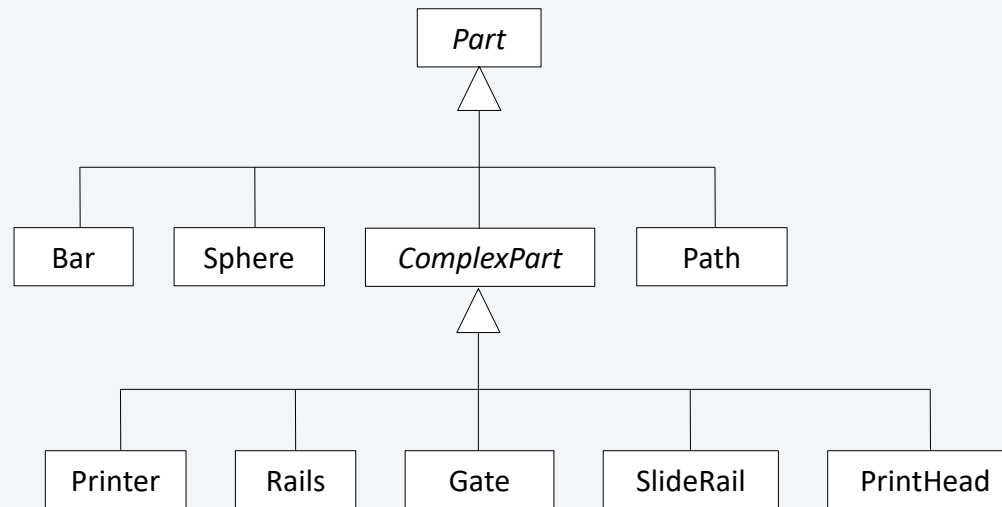
Annahme: Es ist keine Klassenbibliothek verfügbar, bzw. die benötigten Klassen können nicht aus bestehenden Klassen abgeleitet werden

Entwurfsschritt 3: Suche nach geeigneten Basisklassen bzw. Faktorisierung von Gemeinsamkeiten

- Der erste Teil dieses Entwurfsschrittes entfällt, weil keine Klassenbibliothek existiert
- Faktorisierung von Gemeinsamkeiten
 - Alle Objekte müssen gezeichnet werden
 - Fast alle Objekte können bewegt werden
 - Überlegung
 - Alle Objekte können als Teile aufgefasst werden (Abstrakte Basisklasse Part)
 - Weitere Einteilung
 - Es gibt elementare Teile und komplexe Teile (d.h. wiederum aus anderen Teilen zusammengesetzte)
 - Bildung von abstrakten Klassen Part und ComplexPart

Entwurfsschritt 4 und 5: Spezifikationen der Klassen aufstellen und Methoden entwerfen

(1) Klassenbeziehung (Vererbungshierarchie) festlegen



(2) Zuordnung von Messages und Komponenten

- Allgemeine Messages: können für alle Teile (Part) definiert werden

gezeichnet werden

```
drawTop(Rectangle r)
```

```
drawFront(Rectangle r)
```

```
drawRight(Rectangle r)
```

```
drawReal(Rectangle r)
```

Bewegung ausführen

```
moveBy(int dx, int dy, int dz)
```

Warum moveBy statt moveTo?

Objekte haben verschiedene Bezugskoordinaten
relative Bewegung ist vom jeweiligen
Koordinatensystem unabhängig

- (noch) keine Komponentenkandidaten identifiziert!

(3) Konstruktoren festlegen

Klasse	Konstruktor
Part	Part()
ComplexPart	ComplexPart()
Bar	Bar(fromX, fromY, fromZ, toX, toY, toZ, thickness)
Sphere	Sphere(centerX, centerY, centerZ, radius)
Rails	Rails(x0, y0, z0, length, distance)
Gate	Gate(x0, y0, z0, width, height)
SlideRail	SlideRail(x0, y0, z0, width)
PrintHead	PrintHead(x0, y0, z0)
Printer	Printer(x0, y0, z0, width, height, length)
Path	Path(x0, y0, z0)

(4) Detailfragen zur Message-Festlegung

Wie reagieren Rails, Gate und SlideRail auf moveBy?

- Schienen sind unbeweglich: „Leermethode“: moveBy wird ignoriert

„Übersetzung“: `moveBy(dx, dy, dz)`  `moveBy(0, 0, 0)`

- Portal kann sich nur in x-Richtung bewegen

„Übersetzung“: `moveBy(dx, dy, dz)`  `moveBy(dx, 0, 0)`

- Gleitschiene kann sich nur in x- und z-Richtung bewegen

„Übersetzung“: `moveBy(dx, dy, dz)`  `moveBy(dx, 0, dz)`

(5) Klassen ausspezifizieren (Schnittstellen)

```
public abstract class Part {  
    public Part()  
    public abstract void drawTop(Rectangle r);  
    public abstract void drawFront(Rectangle r);  
    public abstract void drawRight(Rectangle r);  
    public abstract void drawReal(Rectangle r);  
    public abstract void moveBy(int dx, int dy, int dz);  
}
```

(5) Klassen ausspezifizieren (Schnittstellen)

```
public abstract class ComplexPart extends Part {
    private final static int MAX_PARTS = 20;
    private int nParts;
    private Part[] partList;
    public ComplexPart() {...}

    -- inherited messages
    public void drawTop(Rectangle r) {...}
    public void drawFront(Rectangle r) {...}
    public void drawRight(Rectangle r) {...}
    public void drawReal(Rectangle r) {...}
    public void moveBy(int dx, int dy, int dz) {...}

    -- new messages
    public void addPart(Part p) {...}
    public Part getPart(int partNr) {...}
    public void removePart(int partNr) {...}
}
```

(5) Klassen ausspezifizieren (Methoden entwerfen)

Klasse ComplexPart

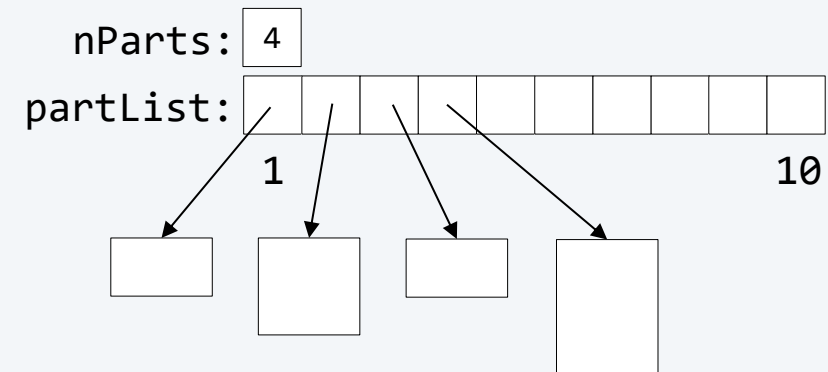
■ Konstruktor

```
public ComplexPart() {  
    this.partList = new Part[MAX_PARTS];  
}
```

■ Methode moveBy

```
public void moveBy(int dx, int dy, int dz) {  
    for(int i = 0; i < nParts; i++) {  
        partList[i].moveBy(dx, dy, dz);  
    }  
}
```

Hinweis: Die dynamische Bindung wird ausgenutzt: Es ist unbedeutend, von welchen Klassen die Elemente im Feld `partList` sind. Siehe auch folgende Methoden.



- Zeichne Grundriss und Aufriss

```
public void drawTop(Rectangle r) {  
    for(int i = 0; i < nParts; i++) {  
        partList[i].drawTop(r);  
    }  
}
```

```
public void drawFront(Rectangle r) {  
    for(int i = 0; i < nParts; i++) {  
        partList[i].drawFront(r);  
    }  
}
```

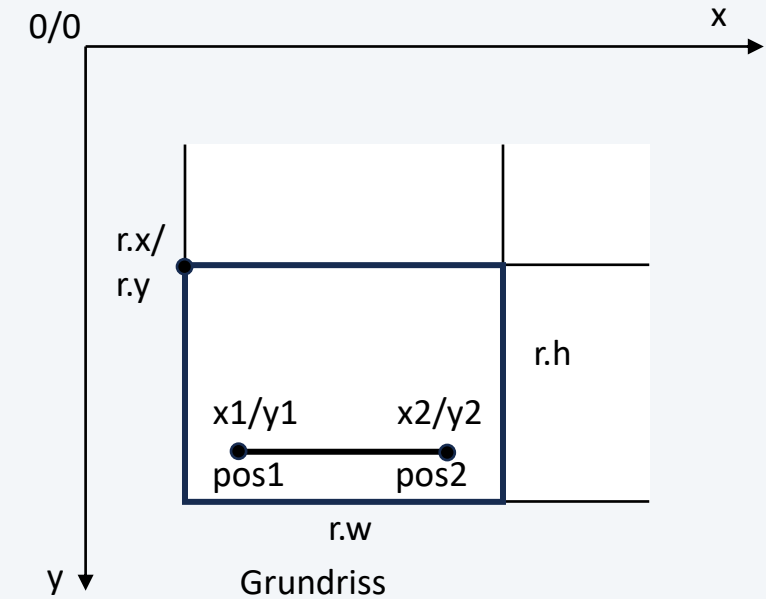
Zeichnen des Seiten- und Schrägrisses analog dazu

Entwurf der Klasse Bar

```
public class Bar extends Part {  
    private final Position pos1;  
    private final Position pos2;  
  
    public Bar(int p1x, int p1y, int p1z, int p2x, int p2y, int p2z) {  
        this.pos1 = new Position(p1x, p1y, p1z);  
        this.pos2 = new Position(p2x, p2y, p2z);  
    }  
    public void moveBy(int dx, int dy, int dz) {  
        pos1.x += dx; pos1.y += dy; pos1.z += dz;  
        pos2.x += dx; pos2.y += dy; pos2.z += dz;  
    }  
    public Position getPosition1() {  
        return new Position(pos1);  
    }  
    public Position getPosition2() {  
        return new Position(pos2);  
    }  
    ...  
}
```

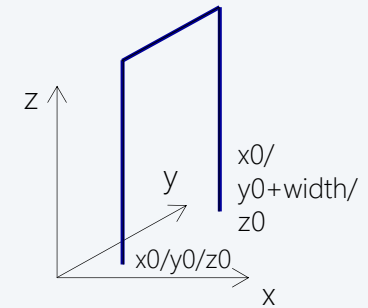

Entwurf der Klasse Bar

```
public class Bar extends Part {  
    ...  
    public void drawTop(Rectangle r) {  
        int x1 = r.x + pos1.x * r.w / 2000;  
        int y1 = r.y + r.h - pos1.y * r.h / 2000;  
        int x2 = r.x + pos2.x * r.w / 2000;  
        int y2 = r.y + r.h - pos2.y * r.h / 2000;  
        StdDraw.line(x1, y1, x2, y2);  
    }  
  
    public void drawFront(Rectangle r) {...}  
    public void drawRight(Rectangle r) {...}  
    public void drawReal(Rectangle r) {...} ...  
}
```



Entwurf der Klasse Gate

```
public class Gate extends ComplexPart {  
    public Gate(int x0, int y0, int z0, int width, int height) {  
        super();  
        Bar b = new Bar(x0, y0, z0, x0, y0, z0+height);  
        this.addPart(b);  
        b = new Bar(x0, y0+width, z0, x0, y0+width, z0+height);  
        this.addPart(b);  
        b = new Bar(x0, y0, z0+height, x0, y0+width, z0+height);  
        this.addPart(b);  
    }  
    public void moveBy(int dx, int dy, int dz) {  
        super.moveBy(dx, 0, 0);  
    }  
}
```



Portal kann sich nur in x-Richtung bewegen
moveBy(...) muss dafür sorgen, dass die
geerbte Methode von ComplexPart mit
modifizierten Parametern aufgerufen wird

Entwurf der Klasse Rails

```
public class Rails extends ComplexPart {  
    public Rails(int x0, int y0, int z0, int length, int distance) {  
        super();  
        Bar front = new Bar(x0, y0, z0, x0 + length, y0, z0);  
        this.addPart(front);  
        Bar rear = new Bar(x0, y0 + distance, z0, x0 + length, y0 + distance, z0);  
        this.addPart(rear);  
    }  
    public void moveBy(int dx, int dy, int dz) {  
        // must not call super.moveBy  
    }  
}
```

Schienen sind unbeweglich: „Leermethode“: moveBy wird ignoriert
super.moveBy(0,0,0); wäre möglich aber wirkungslos!

Entwurf der Klasse Printer

```
public class Printer extends ComplexPart {
    private final Position printPosition;

    public Printer(int x0, int y0, int z0, int width, int height, int length) {
        super();
        Rails rails = new Rails(x0, y0, z0, length, width);
        this.addPart(rails);
        Gate gate = new Gate(x0, y0, z0, width, height);
        this.addPart(gate);
        Slide slide = new Slide(x0, y0, z0, width);
        this.addPart(slide);
        PrintHead head = new PrintHead(x0, y0, z0);
        this.addPart(head);
        this.printPosition = new Position();
    }
    public void moveBy(int dx, int dy, int dz) {
        super.moveBy(dx, dy, dz);
        printPosition.x += dx;
        printPosition.y += dy;
        printPosition.z += dz;
    }
}
```

Wiederverw. der Funktionalität aus ComplexPart

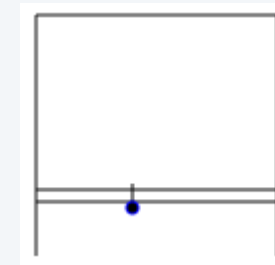
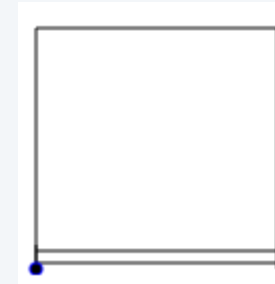
Hinzufügen neuer Funktionalität
(Aktualisieren der Werkzeugposition)

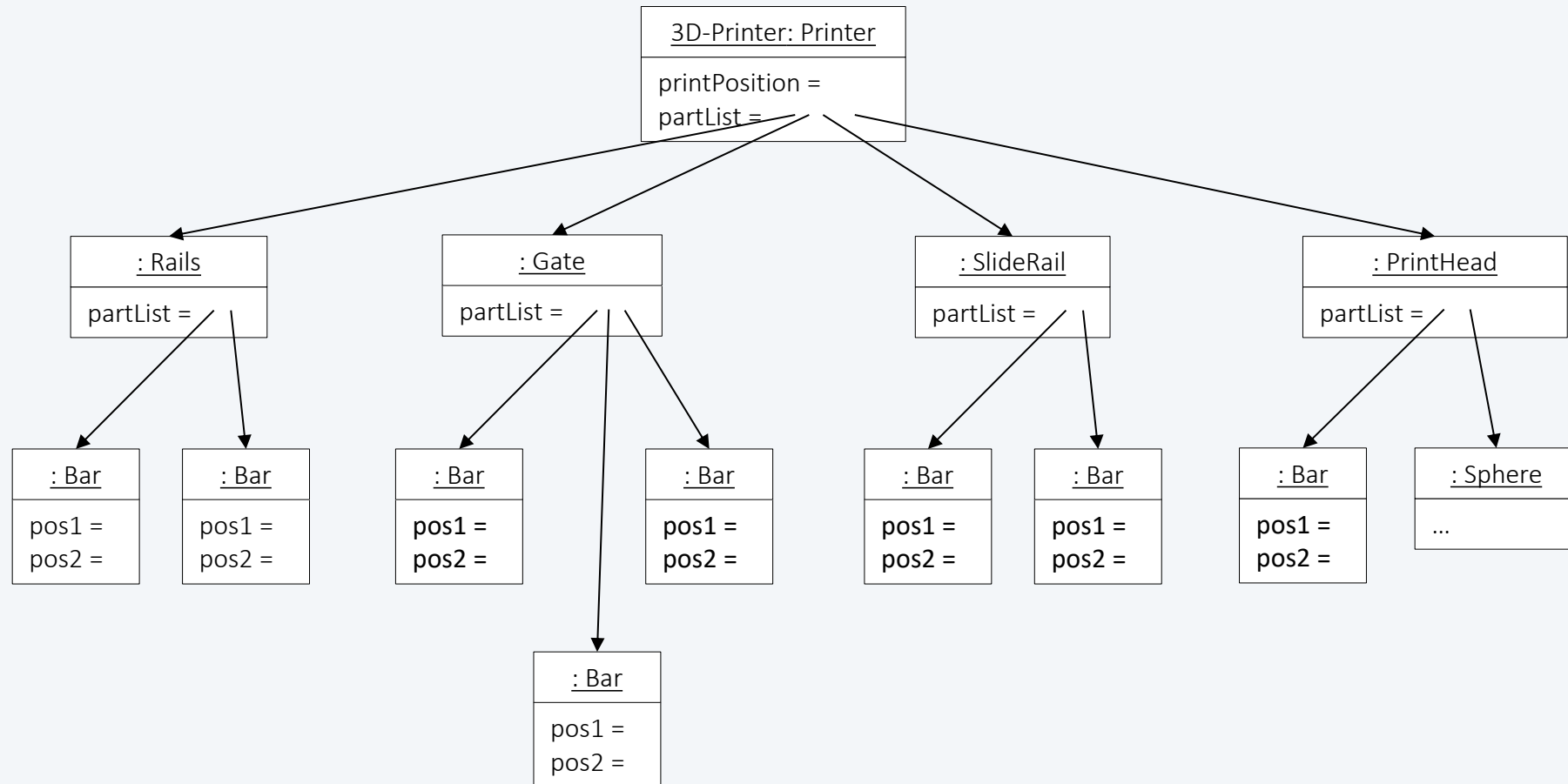
Verwendung der Klasse Printer

```
Printer p;
```

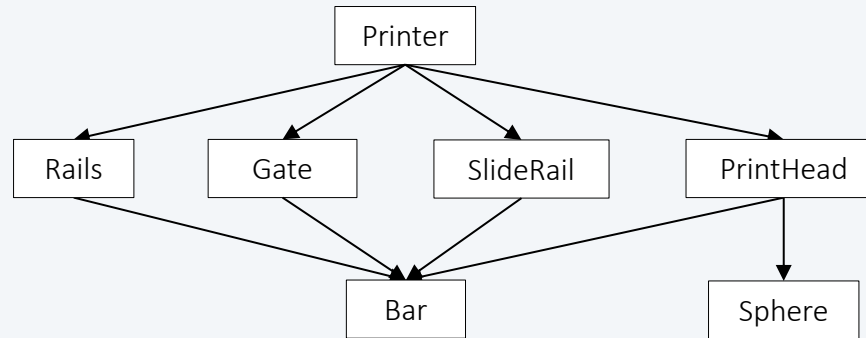
```
p = new Printer(0, 0, 0, 2000, 2000, 2000);  
p.drawRight(...);
```

```
p.moveBy(500, 800, 400);  
p.drawRight(...);
```

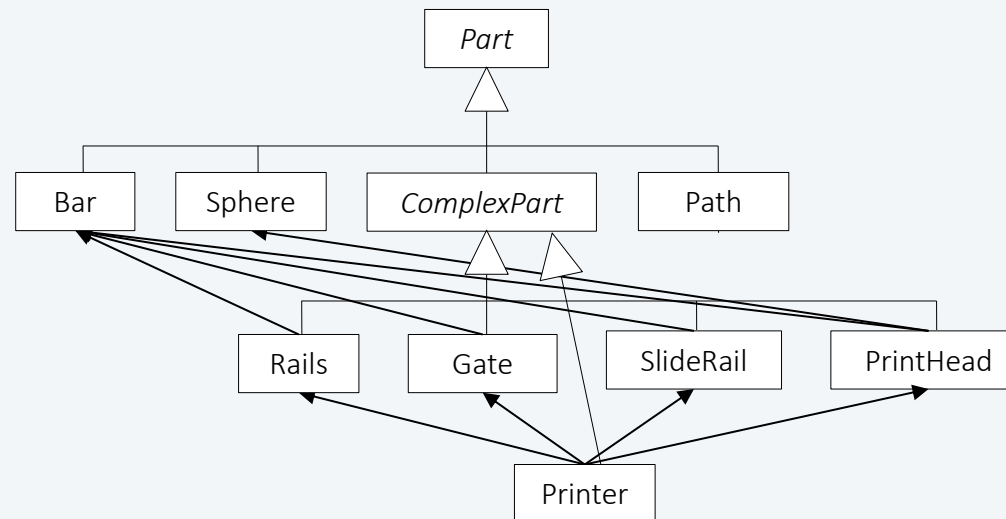




Besteht aus“-Relation



Gemeinsame Darstellung von Vererbungs- („ist-ein“-) und „besteht-aus“-Beziehung



Oft sehr komplexe
Zusammenhänge

Aufgabe: Klassenhierarchie für arithmetische Ausdrücke

Gegeben ist folgende Grammatik für einfache arithmetische Ausdrücke:

```
Expr = Term { '+' Term }.  
Term = Fact { '*' Fact }.  
Fact = number | '(' Expr ')'.  
.
```

Elemente dieser Ausdrücke sind Operanden (Zahlen und geklammerte Ausdrücke) sowie binäre Operatoren (die auf jeweils zwei Operanden angewandt werden). Entwickeln Sie eine Hierarchie von Klassen, deren Objekte es gestatten, arithmetische Ausdrücke in Form von Binärbäumen darzustellen, auszuwerten und in Infix-Notation auszugeben.

Geben Sie zumindest die Klassendeklarationen an, und versuchen Sie dann, die Methodenimplementierungen auszuarbeiten.

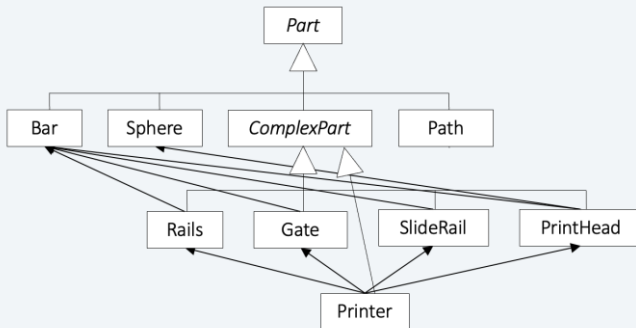
11 Objektorientierter Entwurf

11.1 Grundlegendes u. UML

11.2 Basismodell für oo Systementwurf

11.3 Entwurfsbeispiel 3D-Drucker

11.4 Entwurfsmuster

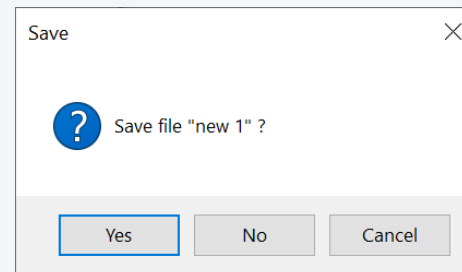
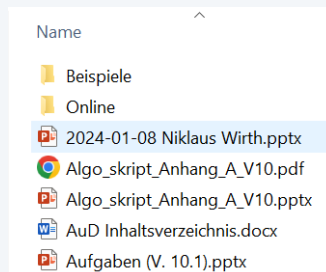
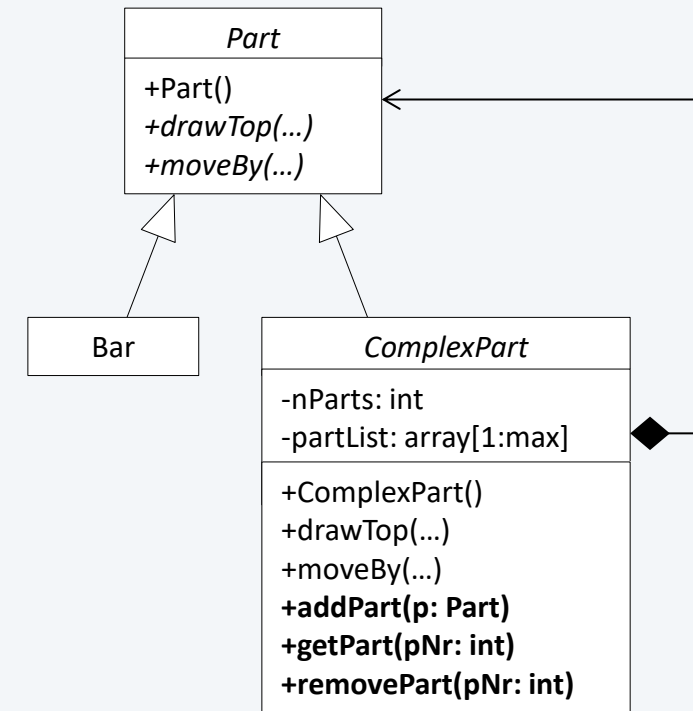


Gutes Design lernt man durch Erfahrung

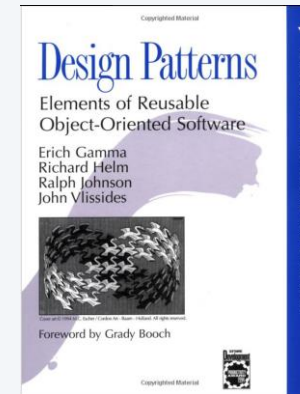
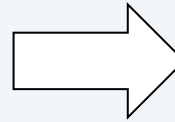
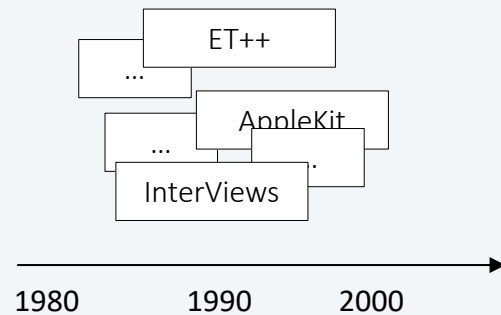
Beispiel: Einfache und komplexe Teile eines 3D-Druckers (Seite 23 u. 24)

- es gibt elementare Teile und komplexe Teile
- Bildung von abstrakten Klassen `Part` und `ComplexPart`

Die hier verwendete Idee, um einfache und komplexe Objekte gleich zu behandeln, finden wir häufig.



Es gibt **Entwurfsmuster** für die Gestaltung von Systemarchitekturen und guten Programmstrukturen



1994

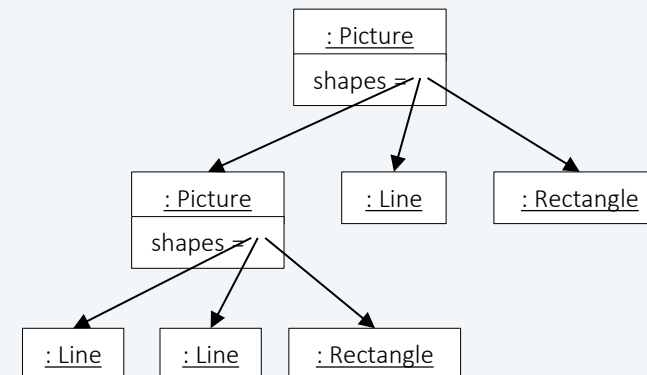
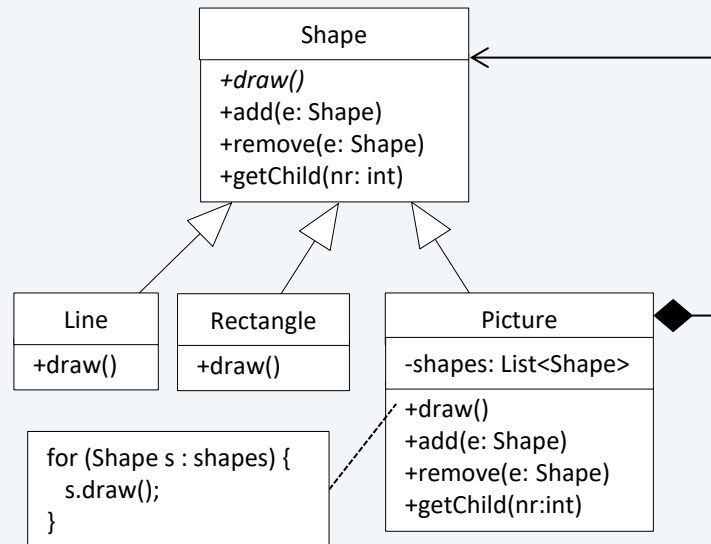
- Katalog mit 23 Entwurfsmuster
- Erzeugungsmuster (engl. *creational patterns*)
- Strukturmuster (engl. *structural patterns*)
- Verhaltensmuster (engl. *behavioral patterns*)

Zweck

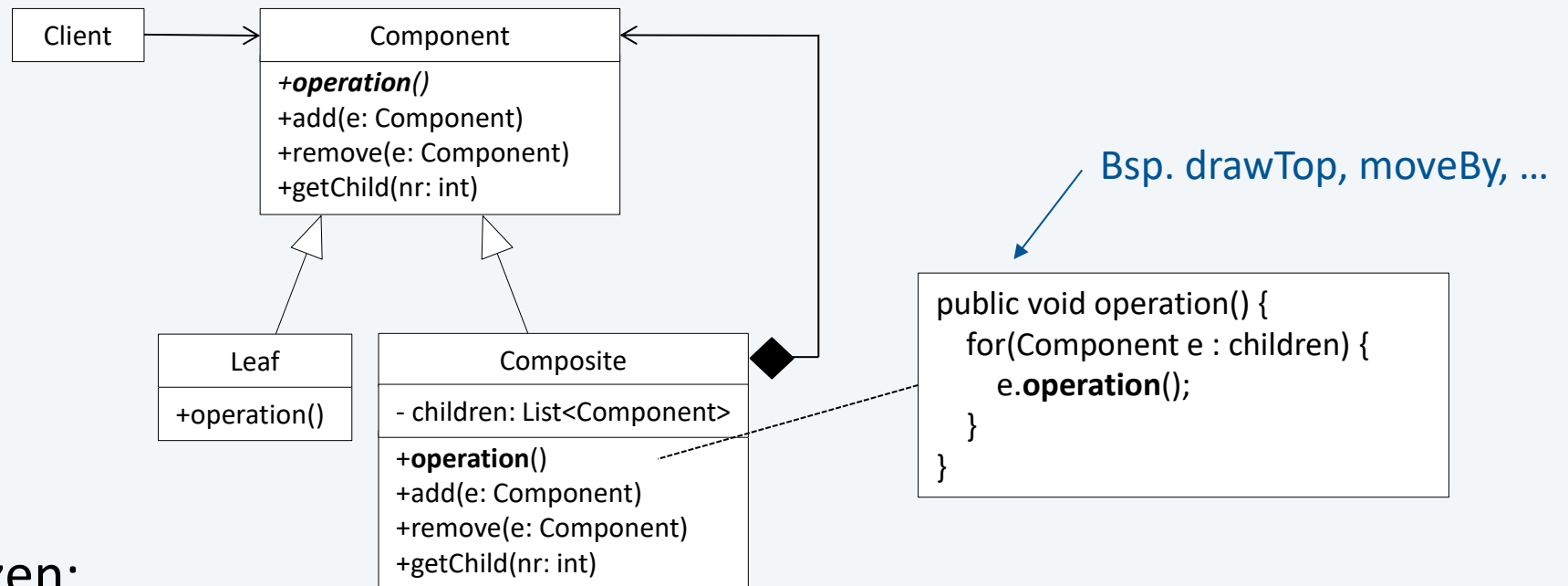
- bilde Teil-Ganzes-Hierarchien von Objekten
- behandle elementare und komplexe Objekte gleich

Motivation

- Grafikeditor mit verschiedenen grafischen Objekten (Linie, Rechteck, ...), die zu Bildern zusammengefasst werden können



Struktur



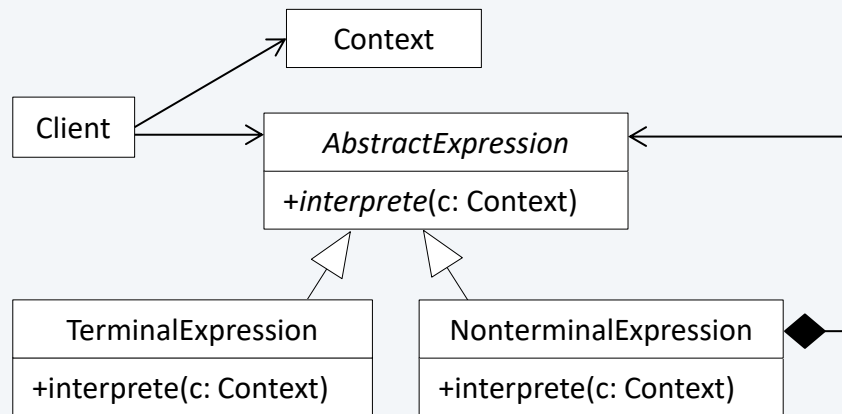
Konsequenzen:

- Verwendung (aus Client-Sicht) wird einfach, neue Komponenten können einfach hinzugefügt werden
- Implementierungsdetails: Verwaltung der Kindknoten
 - Implementierung in der Klasse **Composite**
 - Schnittstelle in **Component**- oder **Composite**-Klasse?

Zweck

- definiere eine Repräsentation für die Grammatik einer formalen Sprache und einen Interpreter, um Sätze dieser Sprache zu interpretieren

Struktur



Objekte der Klassen

TerminalExpression und **NonterminalExpression** bilden einen sogenannten abstrakten Syntaxbaum.

Bestandteile

AbstractExpression deklariert die Operation `interpret`, die für alle Knoten im abstrakten Syntaxbaum implementiert wird

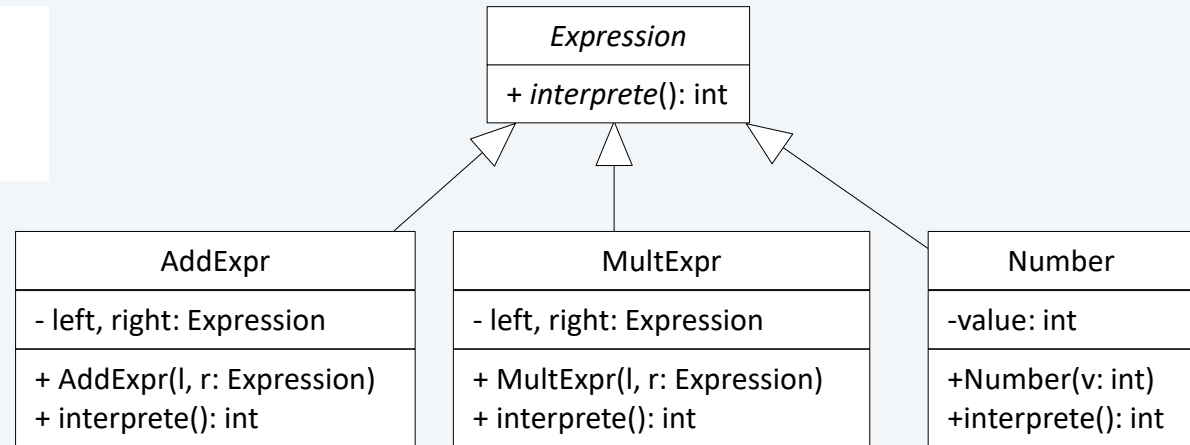
TerminalExpression implementiert die Operation für ein Terminalsymbol (TS) der Sprache. Ein Objekt für jedes TS

NonterminalExpression implementiert die Operation für ein Nonterminalsymbol

Context stellt Informationen (z.B. Variablenspeicher) bereit

Anwendung

```
Expr = Term { '+' Term }.  
Term = Fact { '*' Fact }.  
Fact = number | '(' Expr ')'.  
...
```



Methoden

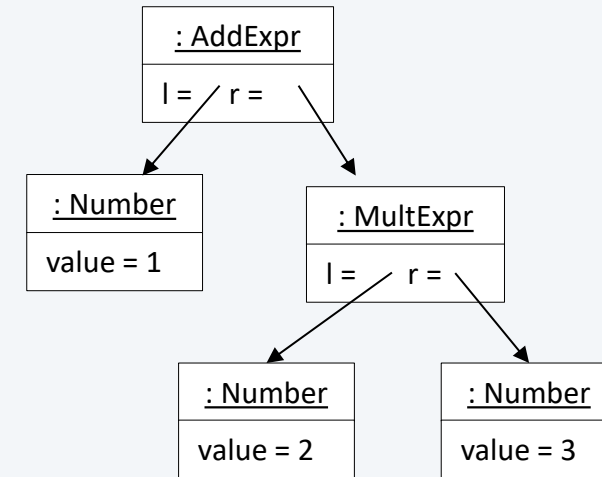
```
public class AddExpr extends Expression {  
    ...  
    public AddExpr(Expression l, Expression r) {  
        this.left = l;  
        this.right = r;  
    }  
  
    public int interpret() {  
        return left.interpret() + right.interpret();  
    }  
}
```

```
public class Number extends Expression {  
    ...  
    public Number(int value) {  
        this.value = value;  
    }  
  
    public int interpret() {  
        return value;  
    }  
}
```

Objektdiagramm für Ausdruck $1 + 2 * 3$

```
Number n1 = new Number(1);  
Number n2 = new Number(2);  
Number n3 = new Number(3);  
MultExpr mul = new MultExpr(n2, n3);  
AddExpr add = new AddExpr(n1, mul);
```

```
int v = add.interprete();
```



Zweck

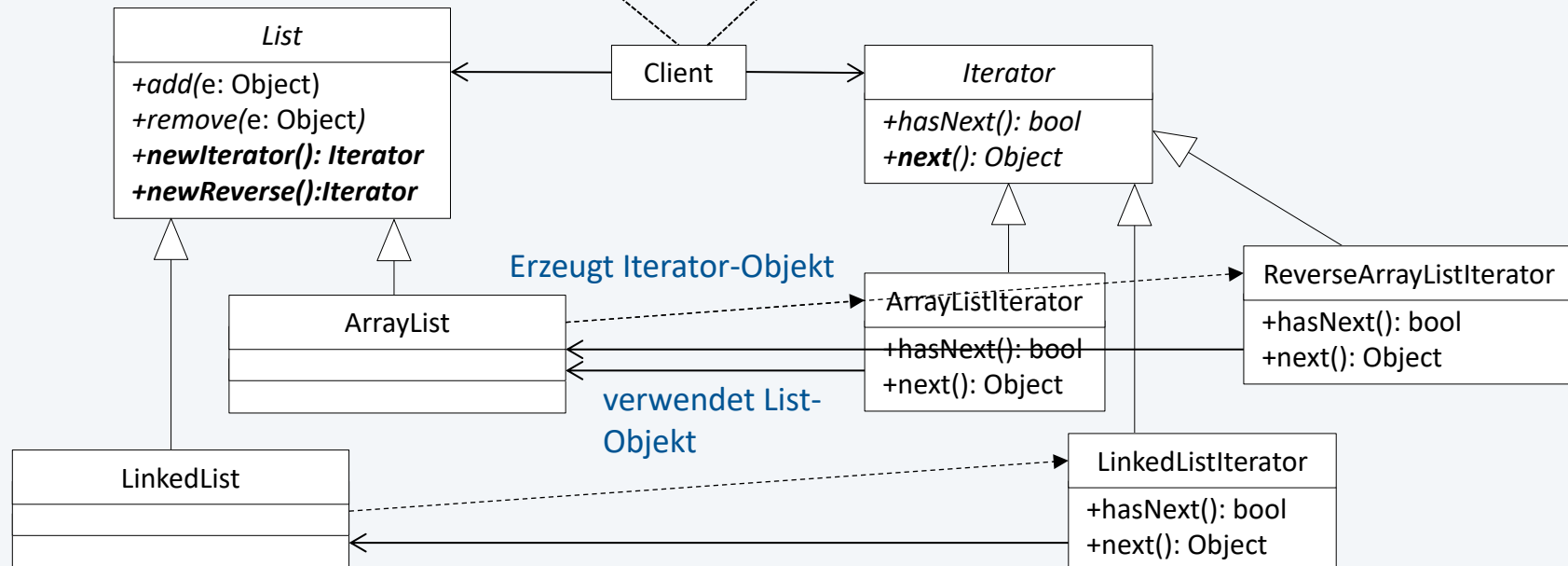
- Möglichkeit, auf die Elemente eines Behälters sequenziell zuzugreifen, ohne interne Darstellung (Datenstruktur) offenzulegen

Motivation

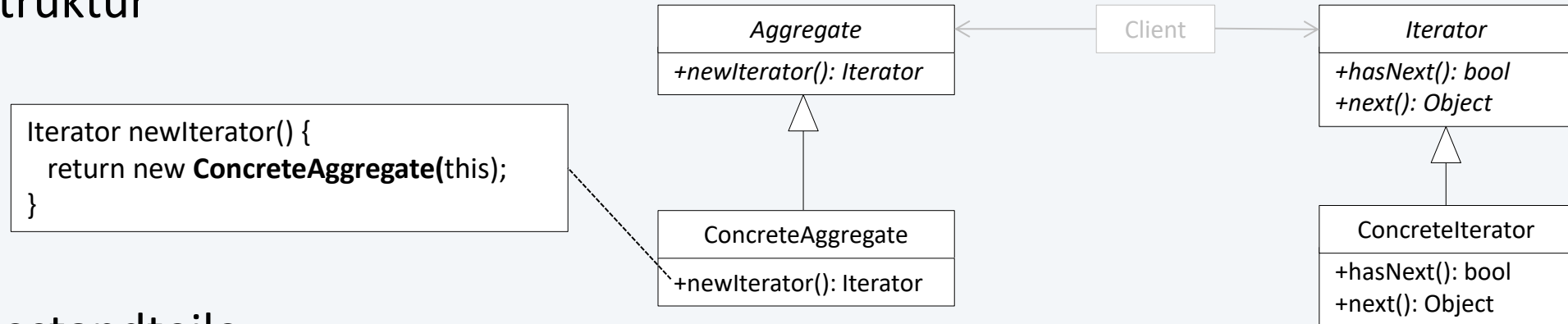
- Zugriff auf Elemente eines Behälters, ohne interne Darstellung offenzulegen
- Behälter auf verschiedene Weise durchlaufen (z.B. vorwärts oder rückwärts)

```
public void forward(List list) {  
    Iterator it = list.newIterator();  
    while (it.hasNext()) {  
        Object obj = it.next();  
        // -- action with obj  
    }  
}
```

```
public void backward(List list) {  
    Iterator it = list.newReverseIterator();  
    while (it.hasNext()) {  
        Object obj = it.next();  
        // -- action with obj  
    }  
}
```



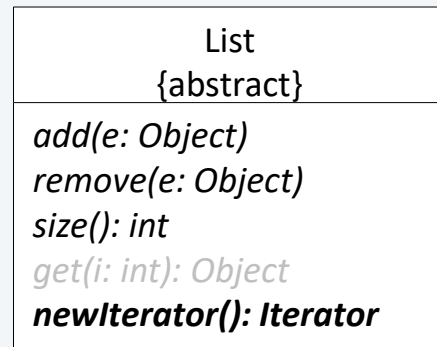
Struktur



Bestandteile

- Iterator definiert eine Schnittstelle für den Zugriff auf Elemente (`next`) und das Durchlaufen von Elementen (`hasNext`)
- ConcreteIterator implementiert Iterator und verfolgt die aktuelle Position beim Durchlaufen des Behälters (Aggregats)
- Aggregate definiert eine Schnittstelle für das Erzeugen eines Iterator-Objekts
- ConcreteAggregate erzeugt und liefert konkrete Iterator-Objekte

Anwendung: Iterator für Liste von Objekten



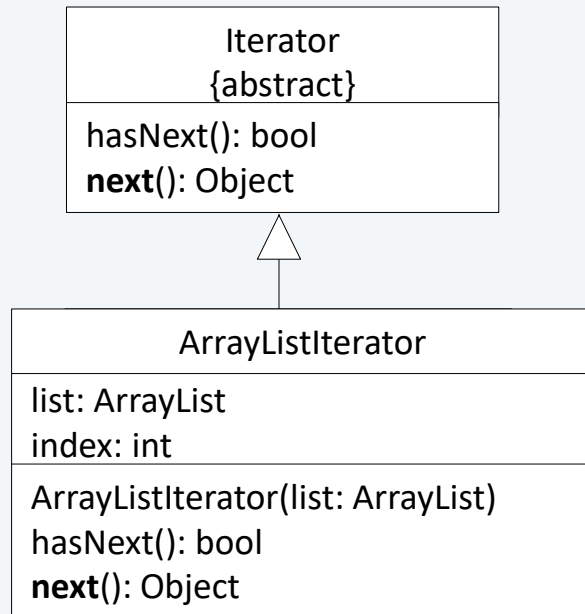
Anmerkung: Zugriff auf Elemente mit Iterator ist allgemeiner als mit Zugriffsoperation `get(i: int): Object`

`ArrayList.get(...)` ... $O(1)$

`LinkedList.get(..)` ... $O(n)$

```
Iterator newIterator() {
    return new ArrayListIterator(this);
}
```

Anwendung: Iterator für Liste von Objekten



```
ArrayListIterator(ArrayList list) {
    this.list = list;
    this.index = 0;
}
```

```
boolean hasNext() {
    return index < list.size();
}
```

```
Object next() {
    Object obj = list.get(index);
    index++;
    return obj;
}
```

Anwendung: Iterator für Liste von Objekten

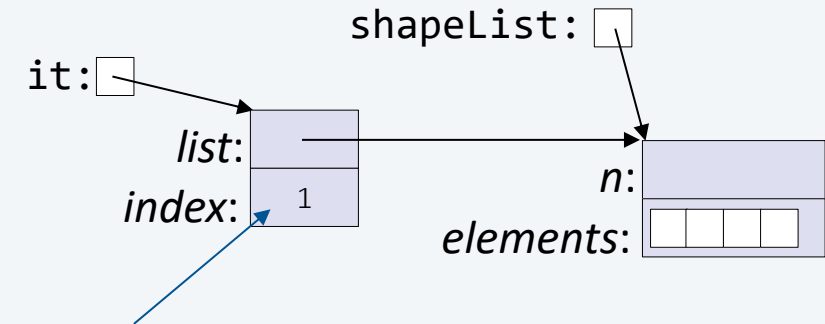
ohne Iterator

```
void drawShapes(List shapeList) {  
    for (int i = 0; i < shapeList.size(); i++) {  
        Object obj = shapeList.get(i);  
        ((Shape)obj).draw();  
    }  
}
```

Aktuelle Position wird in drawShape gehalten

mit Iterator

```
void drawShapes(List shapeList) {  
    Iterator it = shapeList.newIterator();  
    while (it.hasNext()) {  
        Object obj = it.next();  
        ((Shape)obj).draw();  
    }  
}
```



Aktuelle Position wird in Iterator gehalten